



Università
di Catania

Dipartimento di
Ingegneria Elettrica Elettronica e
Informatica

Advanced Programming Languages

Functions and scope

Docente:
Marco Grassia
marco.grassia@unict.it

A.A. 2025/2026

Functions

Functions

- Functions are **building blocks** of modular and reusable code
- They allow you to **encapsulate logic**, improve readability, and reduce duplication
- In C++, functions can be:
 - **Free functions** (global or in namespaces)
 - **Member functions** (inside classes)
 - **Lambda expressions** (anonymous functions)

Free functions

- A function is a named block of code that performs a specific task and can return a value
- Syntax:

```
return_type function_name(parameter_list) {  
    // function body  
}
```

- Example:

```
int add(int a, int b) { return a + b; }
```

C++ parameter passing:

- **Parameter passing** defines how arguments are transmitted to functions
- The choice affects **performance**, **memory usage**, and **mutability** of the data
- In C++, **parameters can be passed by:**
 - **Value (T): copies the argument**, meaning **that data is duplicated** and the original value cannot be modified inside the function. This choice can be **expensive for large objects**
 - **Reference ($T\&$): passes an alias to the original variable**, which is **efficient**. However, **changes inside the function affect the original value**

C++ parameter passing: variations

- **value** (T)
- **reference** ($T\&$)
- **const reference** ($\text{const } T\&$): passes **reference without allowing modification**. Best option for large objects if modifying the original value is not desirable
- **pointer** (T^*): passes by **value the address of the variable**. **Requires explicit dereferencing** ($*\text{ptr}$). Often **used for dynamic memory or optional parameters**, as references cannot be null

Parameter passing

Value

```
int add(int a, int b) {  
    return a + b;  
}
```

```
/* ... */
```

```
int a = 10, b = 20;
```

```
add(a, b);
```

Reference

```
void swap(int &x, int &y) {  
    int temp = x;  
    x = y;  
    y = temp;  
}
```

```
/* ... */
```

```
int a = 10, b = 20;
```

```
swap(a, b);
```

Parameter passing

Const reference

```
int sum(const Point&p) {  
    return p.x + p.y;  
}
```

```
/* ... */  
Point p = { 1, 2};  
sum(p);
```

Pointer

```
void swap(int *x, int *y) {  
    int temp = *x;  
    *x = *y;  
    *y = temp;  
}  
/* ... */
```

```
int a = 10, b = 20;  
  
swap(&a, &b);
```


return

- Terminates the execution of a function and **returns control to the calling function** (or to the operating system if you transfer control from the main function)
- Execution resumes in the calling function at the point immediately following the call
- Syntax:

`return [expression];`
- Three ways to return values to caller

```
int sum(int a, int b) {  
    return a + b;  
}  
  
void f() {  
    std::println("This function does "  
                "not return anything");  
    return;  
}  
  
void f2(int a) {  
    if (a < 0) {  
        std::println("Negative");  
        return;  
    }  
    std::println("Non-negative");  
}
```

return something to caller:

1. by **value**: copy/move the result;
simplest and safest
 2. by **reference**: alias an existing
object
 3. by **pointer**: use when you may
return “no object” (nullptr)
- **Never** return a reference (or pointer)
to a **local** variable: the object is
destroyed at the end of the function
(dangling pointer/ref!)

```
// Return by value
int add(const int &a, const int &b) {
    return a + b;
}

// Return by pointer
int* add_p(const int&a, const int& b) {
    return new int{a+b};
}

// Return by reference
int& max(int &a, int &b) {
    if (a > b)
        return a;
    return b;
}
```

Exercise

- Define the following functions:

- `int& front_ref(int* arr, int n);`
- `int front_val(int* arr, int n);`

usage:

```
int a[] = {7,8,9};  
front_ref(a,3) = 42; // modifica a[0]  
int x = front_val(a,3); // copia
```

- Dangling & lifetime:
 - return the address / reference to a local variable

Arrays as arguments

- In function parameters, an array decays to a pointer to its first element
- The function does not know the size
- The size in the formal arguments is ignored
- You must pass the size as parameter explicitly

```
void sizeof_array(int arr[]) {  
    cout << "Sizeof arr: " << sizeof(arr) << "\n";  
    cout << "(it's a pointer!)\n";  
}
```

Sizeof arr: 8
(it's a pointer!)

```
// Functions with array parameters  
// Equivalent declarations  
int f_arr1(int arr[], size_t size) {}  
int f_arr2(int* arr, size_t size) {}
```

```
// The number of elements is optional and disgarded  
anyway  
int* max_in_array(int arr[10], size_t size) {  
    //...  
}
```

Arrays as arguments

- **Example**

```
int* max_in_array(int arr[], size_t size) {  
    if (size == 0)  
        return nullptr;  
  
    int max_val = arr[0];  
    size_t max_idx = 0;  
    for (size_t i = 1; i < size; i++) {  
        if (arr[i] > max_val) {  
            max_val = arr[i];  
            max_idx = i;  
        }  
    }  
    return &arr[max_idx];  
}
```

Exercise

- Implement:
 - `double average(const int a[], int n);`
 - `void minmax(const int a[], int n, int& out_min, int& out_max);`

(static)

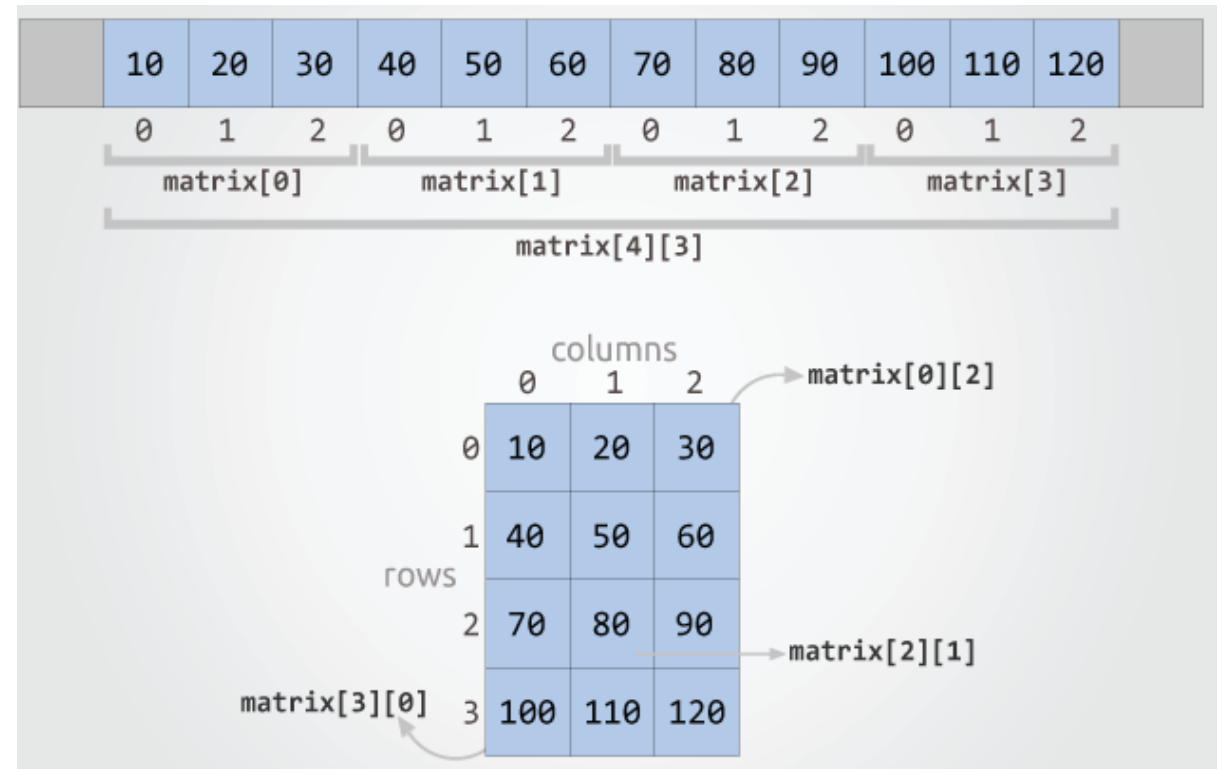
Multidimensional arrays as arguments

- For multidimensional arrays, **you can omit the first dimension only**
- This is because the **compiler needs to know the size of each row to correctly calculate the memory offsets** for accessing elements in the array

```
// For multidimensional arrays,  
// you can omit the first dimension only  
int f_arr3(int arr[][10], size_t rows) {}
```

Multidimensional Arrays in C

- In C, a **static multidimensional array** is essentially an **array of arrays, declared with fixed dimensions known at compile time**
- Memory layout is **row-major order**:
 - All elements of the first row are stored contiguously,
 - followed by all elements of the second row, and so on
- This means that iterating over rows in sequence accesses memory in a cache-friendly manner



Multidimensional Arrays in C

- Accessing elements is straightforward
- Use the **subscript operator [] repeatedly**, specifying each dimension

// Dichiarazione di un array di interi con N dimensioni
`tipo_dati nome_array[dim1][dim2][...][dimN];`

// Dichiarazione di una matrice 2x3
`int matrice[2][3];`

// Accesso all'elemento in posizione i, j
`int elemento = matrice[i][j];`

// Inizializzazione di una matrice 2x3
`int matrice[2][3] = {{1, 2, 3}, {4, 5, 6}};`

Functions:

Declaration and implementation

- The **declaration** (a.k.a. prototype, interface) **introduces the function's name, type and parameter list** to the program
- Examples:

```
return_type function_name(parameter_list);
```

```
int add(int, int);
```

- **Does not require parameter names**
- Usually **placed in header files** (.h, .hpp)

Functions:

Declaration and implementation

- The **implementation provides** both a **declaration** and the **body of the function**
- The return type and the parameter types **must match the declaration (if any)**
- **Parameter names are required**

```
return_type function_name(parameter_list) {  
    // function body  
}  
  
int add(int a, int b) { return a + b; }
```

- Usually placed **in the implementation files** (.c, .cpp)

Exercise

- Declare and then implement:
 - `void scale_by_max(int a[][10], int rows);`

Static variables

- A static local variable:
 - Has **block scope** but **static storage duration** (exists from program start to end)
 - Is **initialized exactly once** (thread-safe since C++11)
 - Preserves its value across calls
- **Use cases:** memoization tables, local singletons
- **Caveat:** global lifetime makes testing/coupling harder

```
int counter() {  
    static int c{0}; // initialized once  
    return ++c;  
}
```

```
void procedure(char c) {  
    static bool first_call = true;  
  
    if (first_call) {  
        // ... Initialization ...  
        first_call = false;  
    }  
    // ... operation on c ...  
}
```

Function overloading (name overloading)

- You can define **multiple functions** with the **same name** but **different parameter types and/or arity** (number of)
- **Signature matters: parameter types and count**
- **Not part of the signature: return type** alone, parameter **names**, **top-level const** (affecting the object, not the ref/pointer)

```
void print(int x) {  
    std::println("Integer: {}", x);  
}  
  
void print(double x) {  
    std::println("Double: {}", x);  
}  
  
print(1);  
print(3.14);
```

Integer: 1
Double: 3.14

Function overloading (name overloading)

- You can define **multiple functions** with the **same name** but **different parameter types and/or arity** (number of)
- **Signature matters: parameter types and count**
- **Not part of the signature: return type** alone, parameter **names**, **top-level const** (affecting the object, not the ref/pointer)

// OK: different parameter types

```
void g(int x) {}  
void g(double y) {}  
void g(float z) {}  
int g(char string[]) {}  
void g(int* px) {}
```

// ERRORS: redefinition of 'g'

// - Top-level const

```
void g(const int x) {}
```

// - Same parameter different name

```
void g(int y) {}
```

// - Different return type

```
int g(int x) {}
```

Overload Resolution (simplified)

- When calling an overloaded function, **the compiler:**
 1. **Builds the set of candidates** (same name, visible)
 2. **Selects viable ones** (parameter count, conversions allowed)
 3. Chooses the **best match** (exact match > promotion > standard conversion > user-defined conversion)
- **In case of ambiguity → compilation error**

```
void print(int x) {  
    std::println("Integer: {}", x);  
}
```

```
void print(double x) {  
    std::println("Double: {}", x);  
}
```

```
print(3.14f); // float  
print('a');  // char
```

Double: 3.140000104904175

Integer: 97

Overloading exercise

- Implement and overload a sum function for arrays:
 - `int sum(const int* first, const int* last);` *// pointer-range*
 - `int sum(const int a[], int n);` *// array decays to pointer*

lvalues and *rvalues* discussion

- A **non-const lvalue reference (T&)** cannot bind to an rvalue (temporary)
- A **const lvalue reference (const T&)** *can* bind to an rvalue
- **Without this rule**, every time you passed a **temporary** to a function, it **would require a copy as it is a soon-to-die object**
- Allowing const T& avoids that extra copy (it binds directly to the temporary)

```
// by value  
void f(std::string s);  
// constructs temporary, then copies into s  
f("hello");  
  
// temporary lives as long as ref  
const std::string& ref = std::string("hi");  
  
void f(const std::string& s);  
f("hello"); // binds to const lvalue ref
```

lvalues and *rvalues* discussion

- The **reference** is **const**, so the **function cannot modify the temporary** in a way that would surprise the caller
- If it were T&, you could modify a temporary that disappears right after the call, which is meaningless
- For functions that want to **take ownership** of a temporary, use **rvalue references (T&&)**

```
// by value  
void f(std::string s);  
// constructs temporary, then copies into s  
f("hello");  
  
// temporary lives as long as ref  
const std::string& ref = std::string("hi");  
  
void f(const std::string& s);  
f("hello"); // binds to const lvalue ref
```

Overload resolution example (1)

- *print(a)*:
 - `int` → `int` (exact match) for first overload
 - `int` → `const int&` (binding to lvalue) for second overload
 - Both are viable, but **exact match beats reference binding** → calls `print(int)`
- *print(42)*:
 - `int` → `int` (exact match) for first overload
 - `int` → `const int&` (binds to temporary) for second overload
 - Still exact match → calls `print(int)`
- **If we remove `print(int)`**, then `print(const int&)` handles both lvalues and rvalues
- Adding a by-value overload often “shadows” a const-ref overload for cheap types

```
void print(int x) {  
    std::cout << "by value\n";  
}  
void print(const int& x) {  
    std::cout << "by const ref\n";  
}  
  
int main() {  
    int a = 10;  
    print(a); // which one?  
    print(42); // which one?  
}
```

Overload resolution example (2)

l-values and r-values

- What happens if we do not have the r-value overload?
- Then, what happens if we remove the `const int&`?

```
void h(int& x) {  
    std::println("h called with lvalue reference: {}", x);  
}  
void h(const int& x) {  
    std::println("h called with const lvalue reference: {}", x);  
}  
void h(int && x) {  
    std::println("h called with rvalue reference: {}", x);  
}  
  
int q = 10;  
const int &q_ref = q;  
  
h(1);  
h(q_ref);  
h(q);
```

h called with rvalue reference: 1

h called with const lvalue reference: 10

h called with lvalue reference: 10

Overload resolution

Takeaways

- **Exact match wins over reference** binding
- Adding a **by-value overload** often “**shadows**” a **ref overload** for cheap types
- **non-const lvalue refs cannot bind to temporaries or const objects**
- For large objects, **prefer *const T&* to avoid copies**

```
void h(int& x) {  
    std::println("h called with lvalue reference: {}", x);  
}  
void h(const int& x) {  
    std::println("h called with const lvalue reference: {}", x);  
}  
void h(int && x) {  
    std::println("h called with rvalue reference: {}", x);  
}  
  
int q = 10;  
const int &q_ref = q;  
  
h(1);  
h(q_ref);  
h(q);
```

h called with rvalue reference: 1
h called with const lvalue reference: 10
h called with lvalue reference: 10

Overloading exercise

- Implement and overload a function to:
 - Accept values of a T
 - Accept constant values $\text{const } T$
 - Accept temporaries $T\&\&$
 - Accept constant ref values $\text{const } \&T$
- How many functions do actually you need?

Functions: default arguments

- Allows a **function to be called without** providing **one or more trailing arguments**
- Indicated by using the **following syntax** for a parameter in the parameter-list **of a function declaration** (typically in the header)
`type param_name = default_value`
- Default arguments **can cause overloading ambiguities**

```
void point(int x = 3, int y = 4);
```

```
point(1, 2); // calls point(1, 2)  
point(1);   // calls point(1, 4)  
point();    // calls point(3, 4)
```

```
// Error: only the trailing parameters can  
// have default arguments (assuming there's  
// no previous declaration of "x")
```

```
int x(int);  
int x(int = 1, int);
```

```
int x(int);  
int x(int, int=3);
```

```
x(1); // which should be invoked??
```


Functions: default arguments

- Defaults are **not** part of the signature (they don't distinguish overloads)
- **Specify the default once** in a single declaration, no need to repeat in multiple declarations
- Be careful mixing defaults and overloading (or avoid mixing)

```
void defaults_example(int a, int b = 5) {  
    std::println("a: {}, b: {}", a, b);  
}  
void defaults_example(int a, double b=2.5) {  
    std::println("a: {}, b: {}", a, b);  
}  
defaults_example(1); // still ambiguous
```

```
void f(int n, int k = 1);  
void f(int n = 0, int k);  
// OK: the default argument of "k" is provided by  
// the previous declaration in the same scope
```

```
void f(int n = 1, int k = 5);  
// ERROR : redefinition of default argument  
// for "n" and "k"
```

Function pointers

- Function pointers **store the address of a function** to call it indirectly
- Syntax:
`ret_type (*fp) (type param, ...);`
- Address-of a function is implicit; `fp` and `&fp` are equivalent in calls
- **No arithmetic allowed** on function pointers

```
void g(int);

void (*g_ptr)(int) = g;
void (*g_ptr2)(int) = &g;

g_ptr(1);
g_ptr2(1);

(*g_ptr)(1);
(*g_ptr2)(1);
g_ptr++; // ERROR: cannot increment
         // function pointer
```

Function pointers exercise

- Implement the following function:
 - `double accumulate(double a[], size_t size, double (*op)(double, double), double init = 0.0);`

inline keyword

- The ***inline*** keyword is a **suggestion** to the compiler for **inlining**, meaning that the **function invocations are replaced with the body of the function in-line**
- This **avoids function call overhead**
- Used for **small, frequently called functions**
- Sometimes compared to old-style C macros, but offer type safety and static checks

```
inline int sum (int a, int b) {  
    return a + b;  
}
```

```
int uses_sum() {  
    return sum(1, 2) * 2;  
}
```

// Becomes after inlining:

```
int uses_sum() {  
    return (1 + 2) * 2;  
}
```

inline keyword

- *inline* can cause issues with recursive functions
- For instance, iterative replacements would cause the code to grow and become huge
- Not a performance guarantee. The compiler (optimizer) decides whether to inline or not

```
inline int sum (int a, int b) {  
    return a + b;  
}
```

```
int uses_sum() {  
    return sum(1, 2) * 2;  
}
```

// Becomes after inlining:

```
int uses_sum() {  
    return (1 + 2) * 2;  
}
```

Lambda expressions

- Lambdas are **anonymous functions** that can be **defined inline and assigned to variables or passed as arguments**
- Introduced in C++11, they are especially useful for **functional-style programming and STL algorithms** (we'll come back to that later)

Syntax:

```
[<capture>] (<parameters>) -> return_type {  
    /* function body */  
};
```

Example:

```
[](int a, int b) -> int {  
    return a + b;  
};
```

Lambda expressions

- **Capture specifies which variables from the surrounding scope are accessible. Cannot be omitted**
- **Parameter list: just like any free function**, can also be empty or omitted!
- **Return type: optional if deducible**
- **Function body: the actual logic**

Syntax:

```
[<capture>] ([<parameters>]) [-> return_type] {  
    /* function body */  
};
```

Examples:

```
[](int a, int b) -> int {  
    return a + b;  
};
```

// Here a and b are global variables

```
auto add_lambda = [] -> int {  
    return a + b;  
};
```

Lambda expressions: *capture* specification

- **Capture** specifies which variables from the surrounding scope are accessible
- Capturing **specific variables or combinations are also possible** (e.g., only x, only x by reference, all by value but x by reference, and so on)

Syntax	Captures...
[]	... no variables
[=]	... all variables by value
[&]	... all variables by reference

Syntax	Captures...
[x]	... x by value
[&x]	... x by reference
[=, &x]	... all by value, x by reference
[&, x]	... all by reference, x by value

Lambda expressions: examples

- **Capture specifies which variables from the surrounding scope are accessible**

- Syntax:
[<capture>] (<parameters>) ->
return_type {
 / function body */*
};

Syntax	Captures...
[]	... no variables
[=]	... all variables by value
[&]	... all variables by reference

```
int a = 10, b = 20;

auto f1 = [a] {
    return a * 2;
};

auto f2 = [a, b](double m) {
    return a * m + b;
};

auto f3 = [&a, b](double m) {
    a += 1;
    return a * m + b;
};

auto f4 = [=](double m) -> auto {
    return a * m + b;
};

double offset = 0.5;
auto f5 = [=](double m) {
    return a * m + b + offset;
};
```

Function pointers exercise

- Implement the following function:
 - `double accumulate(double a[], size_t size, double (*op)(double, double), double init = 0.0);`
- Use a lambda as default parameter and provide an op function using lambdas

From run-time to compile-time

- In order **to improve performance, safety** (early errors), and **expressiveness** (usable in contexts requiring constant expressions)...
- ... **move work from run time to compile time** when possible
- Tools:
 - **constexpr**: *can* be evaluated at compile time (if given constant inputs)
 - **consteval**: *must* be evaluated at compile time (immediate functions)

constexpr (C++11)

- ***constexpr*** (short for ***constant expression***) is a **keyword** that **indicates** that the **value of a variable or the result of a function can be evaluated at compile time**
- **Improves runtime performance** by shifting computation to compile time
- Enables **use of values in contexts that require constant expressions** (e.g., array sizes, template parameters)
- Encourages safer and more predictable code
- **All arguments** to functions must be **constant expressions**
- ***constexpr* functions** cannot contain runtime-only constructs (e.g., *new*, *try*, *virtual*)

constexpr variables

- `constexpr` variables are **implicitly const** and require an **immediately evaluable initializer**
- For constants in headers: use *inline constexpr* (C++17+) instead of *#define*

```
// OK: constant expression
constexpr int MaxSize = 1024;
// OK
constexpr double Pi = 3.141592653589793;

// ERROR: n is not constant
int n = 42;
constexpr int Bad = n;
```

constexpr

- Declaring a **constexpr function** means that it is **eligible** for compile-time
- Note that **non-constexpr functions are still eligible for precomputation and other optimizations** by the compiler
- constexpr functions are implicitly *inline*

```
constexpr int square(int x) {  
    return x * x;  
}
```

// Evaluated at compile time

```
int value = square(5);
```

// OK, run-time (y not constant)

```
int y = 7;
```

```
int B = sqr(y);
```

```
struct Point {  
    int x, y;  
    constexpr Point(int a, int b) : x(a), y(b) {}  
    constexpr int sum() const { return x + y; }  
};  
constexpr Point p(3, 4);  
constexpr int total = p.sum(); // total = 7
```

Exercise

- Implement the Fibonacci sequence:
- `constexpr unsigned fib(unsigned n);`
- `fib(0) == 0`
`static_assert(fib(1) == 1);`
- `fib(1) == 1`
`static_assert(fib(0) == 0);`
- Works in `static_assert(fib(10) == 55);` (break it using a wrong value)
- Note how it is also callable with a run-time int

constexpr (C++20) immediate functions

- `constexpr` == **mandatory** compile-time:
- It **specifies** that a function is an ***immediate function***, that is, **every call to the function must produce a compile-time constant**
- A `constexpr` function must be evaluated **at compile time**; any call at run time is **ill-formed**
- Great for **validation** and **code generation** at compile time

```
constexpr int sqr(int n) {  
    return n * n;  
}
```

```
constexpr int r = sqr(100); // OK
```

```
int x = 100;
```

```
// Error: Call does not produce a constant
```

```
int r2 = sqr(x);
```

```
constexpr int sqrsqr(int n) {
```

```
    // Not a constant expression at this point, but OK
```

```
    return sqr(sqr(n));
```

```
}
```

```
constexpr int dblsqr(int n) {
```

```
    // Error: Enclosing function is not constexpr
```

```
    return 2 * sqr(n);
```

```
    // and sqr(n) is not a constant
```

```
}
```


constexpr vs consteval

- Use `constexpr` when you want **flexibility**: the same function works at **compile time and run time**.
- Use `consteval` when you want **guaranteed compile-time** (e.g., validating template parameters, building lookup tables, enforcing constraints)
- Prefer `inline constexpr` for **header constants**

Scope

APL: C++

Concept refresher

- **Scope:** the **region** of the program **where a name** (identifier) **can be used**. In C++ scope is **static (lexical)**
- **Visibility:** **whether a name is accessible** at a given point in the code (depends on scope and name lookup rules)
- **Lifetime:** **the time during program** execution when an object exists and can be used
- **Storage duration:** **how long an object exists in memory**; determines how long storage for an object is reserved (automatic, static, thread-local, dynamic)
- **Linkage:** whether **two declarations of the same name in different scopes** or **translation units refer to the same entity**

Scope categories in C++

- **Global scope:** names declared **outside any class, function, or namespace**
- **Namespace scope:** names declared **inside a namespace**
- **Block scope:** names declared **inside {}**
- **Local scope (informal term):** names **inside a function** (parameters + block-scoped variables)
- **Statement scope:** names declared **in a for, if, while, or switch statement** are visible **until the end of the statement block**
- **Class scope:** members of a class are visible **inside the class definition**
- **Function scope (rare):** labels for *goto* have function scope

Global scope

- A **global name** is one that is **declared outside of any class, function, or namespace**
- **In C++** even these names **exist with an implicit global namespace**
- The **scope** of global names **extends from the point of declaration to the end of the file** in which they are declared (and possibly other Translation Units due to external linkage)
- **static storage duration** by default: their storage is allocated during program start-up and released after program termination

Visibility rules

- Visibility determines **which declaration is found during name lookup**
- **Lookup order for unqualified lookup searches:**
 - **Current block scope outward**
 - **Enclosing scopes** (function, class, namespace)
 - **Global namespace**
 - Argument-Dependent Lookup (ADL) for functions
- ***Using-directives/using-declarations* affect lookup but do not create new scopes**

```
int g = 10; // global
int h() {
    int i = 20; // local to h

    {
        for (int j = 0; j < 5; j++) // local to loop
            std::print("j: {}\\n", j);

        int p = 30; // local to block
        {
            print("i: {}, p: {}\\n", i, p);
        }
    }

    return g + i;
}
```

Shadowing

- Inner declarations hide outer ones with the same name
- **For global variables, use the scope resolution operator `::` to access names in the global namespace**
- **For namespaces, use fully qualified names**

```
int x = 1; // global
void f() {
    int x = 2; // shadows global
    {
        int x = 3; // shadows outer x
        std::cout << x; // 3

        std::cout << ::x; // 1
    }
    std::cout << x; // 2
}
```

Shadowing

- Inner declarations hide outer ones with the same name
- For namespaces: use fully qualified names
- For global variables, use the scope resolution operator ::

```
namespace Outer {  
    int value = 100;  
  
    namespace Inner {  
        int value = 200;  
  
        void print_values() {  
            // prints 200  
            print("Inner value: {}\n", value);  
            print("Inner value: {}\n", Inner::value);  
            // prints 100  
            print("Outer value: {}\n", Outer::value);  
        }  
    }  
}
```


Object instantiation

In C++, where you put an object determines **storage duration** and **lifetime**

- **Automatic storage (stack):** local variables; destroyed at scope exit
- **Static / thread storage:** globals, static, thread_local; lifetime = program/thread
- **Dynamic storage (free store):** via new/allocators; lifetime ends at *delete*

Storage durations: automatic storage (block scope)

- For automatic **non-static variables**, **scope** also **determines when they are created and destroyed** in program memory
- Allocated on entering block, **freed at block exit**
- Destructed in **reverse construction** order
- **Typical for local variables**
- **Default value is garbage**

```
{ // new scope  
  // automatic storage duration for local vars  
  int x = 30;  
  double y = 3.14;  
} // x and y destroyed here  
  
void f() {  
  int x = 42; // automatic storage duration  
} // x destroyed here
```

Storage durations:

Static storage

- Exists **for entire program** (from before main starts to after it ends)
- **Includes namespace-scope objects, static data members, function-local statics**
- Initialization might be **constant** or **dynamic** (we'll discuss about this later)
- Allocated during program start-up and released after program termination
- “Global scope” (i.e., namespace scope) has **static storage duration**

```
// static storage duration  
int global = 1;  
  
void f() {  
    // persists across calls  
    static int counter = 0;  
    ++counter;  
}
```

Storage durations:

thread storage (thread_local)

- One **instance per thread**
- **Each thread performs initialization of its instance on first use** (or at thread start for some forms); **destroyed at thread exit**
- Function-local static thread_local is initialized **per-thread on first entry**

```
thread_local int tls = 0; // one instance per thread  
  
void bump() { ++tls; }
```

Storage durations: Dynamic storage

- Obtained via new/allocators; released via delete/allocator
- Lifetime is **independent** from the pointer's scope
- Beware **ownership** and **exception safety**

```
float z = new float(2.71f); // dynamic storage duration  
delete z; // free memory
```

```
int* array = new int[10]; // dynamic storage duration  
delete[] array; // free memory
```

Storage duration: Summing up

```
#include <string>
static std::string g_name = "api";    // static storage, dynamic init

int main() {                          // automatic scope begins
    std::string local = "session";    // automatic storage
    static int counter = 0;           // static storage, initialized on first entry
    ++counter;

    auto* p = new std::string("heap"); // dynamic storage, lifetime until delete
    delete p;                          // end of lifetime for *p
} // locals destroyed here, in reverse order
```